Entregable: Ítem 4 - Cinemática Inversa :

1. Fundamento Teórico: Delegación y Auditoría:

En esta etapa no se implementó un algoritmo propio de cinemática inversa (IK); en su lugar, la resolución de las posiciones cartesianas se confió al firmware del manipulador a través de la función send_coords() de la librería pymycobot. Por lo tanto, la meta principal de esta sección consiste en emplear la formulación de Cinemática Directa (FK) verificada en el Ítem 1 como un sistema de auditoría capaz de evaluar la exactitud del algoritmo del fabricante.

2. Implementación en Código

Se construyó el script auditar_ik.py para automatizar la auditoría. El flujo de ejecución es el siguiente:

- Inicialización del sistema: conexión con el JetCobot y retorno a la postura inicial de referencia (todos los ejes articulares en cero).
- Comando de posición objetivo: envío de un punto cartesiano de destino (por ejemplo, x=200.0, y=-100.0, z=150.0) con orientación fija mediante la rutina send_coords().
- Lectura del estado real: captura de las coordenadas articulares alcanzadas e informadas por el firmware del controlador usando get_angles() al concluir la trayectoria.
- Auditoría cinemática: conversión de los ángulos a radianes e ingreso de estos valores en la función personalizada de Cinemática Directa fk(q).
- Cálculo de desviación: determinación del "Error Cartesiano Total" mediante el cálculo de la distancia euclidiana tridimensional entre la posición consignada y la obtenida a través del modelo FK.

```
#!/usr/bin/env python3
"""Ítem 4 — La FK que construimos audita la cinemática inversa del firmware."""
import argparse
import csv
import datetime
import json
import math
import os
import sys
import time

AQUI = os.path.dirname(os.path.abspath(__file__))
for _ruta in (os.path.join(AQUI, '..', 'src', 'arm_broker'), AQUI):
    if _ruta not in sys.path:
        sys.path.insert(0, _ruta)
try:
    from arm_broker import fk
```

```
except ImportError:
    import fk
import ik_ramas as ik
from brazo_util import conectar, dist, esperar_reposo, leer
from mover_a_base import ir_a_base

OBJETIVO_DEFECTO = (200.0, -100.0, 150.0)
ORIENT_DEFECTO = (-180.0, 0.0, 0.0)
LINEA = '=' * 72


def predecir(objetivo, orient, q_base, semillas):
    T = ik.pose_T(objetivo, ik.rpy_a_R(*orient))
    sols = ik.buscar_soluciones(T, semillas=semillas)
    return [ik.describir(q, q_base) for q in sols]


def main():
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawTextHelpFormatter)
    ap.add_argument('--x', type=float)
    ap.add_argument('--y', type=float)
    ap.add_argument('--z', type=float)
    ap.add_argument('--orientacion', type=float, nargs=3, metavar=('RX', 'RY',
'RZ'),
                    default=list(ORIENT_DEFECTO), help='grados (por defecto -180
0 0)')
    ap.add_argument('--puerto', default='/dev/ttyUSB0')
    ap.add_argument('--baud', type=int, default=0, help='0 = probar 1000000 y
115200')
    ap.add_argument('--velocidad', type=int, default=30)
    ap.add_argument('--modo', type=int, default=0, choices=(0, 1),
                    help='send_coords: 0 = angular (por defecto), 1 = lineal')
    ap.add_argument('--semillas', type=int, default=60, help='semillas del
análisis de ramas')
    ap.add_argument('--sin-ramas', action='store_true', help='omitir el análisis
de ramas')
    ap.add_argument('--solo-planear', action='store_true', help='predecir y
salir, sin mover')
    ap.add_argument('--salida', default='auditoria_ik.csv')
    ap.add_argument('--json', default='auditoria_ik.json')
    ap.add_argument('--no-volver', action='store_true', help='no regresar a la
base al final')
    ap.add_argument('--si', action='store_true', help='no pedir confirmaciones')
    args = ap.parse_args()

    if (args.x, args.y, args.z).count(None) not in (0, 3):
        sys.exit('Da --x --y --z juntos, o ninguno.')
    objetivo = [args.x, args.y, args.z] if args.x is not None else
list(OBJETIVO_DEFECTO)
    orient = list(args.orientacion)
    r = math.sqrt(sum(v * v for v in objetivo))
    print(LINEA)
    print(f'Objetivo: x={objetivo[0]:.1f}  y={objetivo[1]:.1f}
z={objetivo[2]:.1f} mm   '
          f'orientación={orient}   (distancia a la base {r:.0f} mm)')
    problemas = []
    if r > fk.ALCANCE_MAX_MM:
        problemas.append(f'está a {r:.0f} mm de la base; el alcance máximo es
{fk.ALCANCE_MAX_MM:.0f} mm')
    if r < fk.ALCANCE_MIN_MM:
        problemas.append(f'está a {r:.0f} mm de la base: demasiado cerca')
    if objetivo[2] < 40.0:
        problemas.append(f'z = {objetivo[2]:.0f} mm: muy cerca de la mesa;
empieza más alto (≥ 40 mm) ''y recuerda que la pinza/herramienta alarga el
brazo')
    for p in problemas:
```

```
        print(f'  ¡CUIDADO! {p}')
    if problemas and not args.si and not args.solo_planear:
        input('  ENTER para seguir igualmente, Ctrl+C para cancelar... ')

    print('\nPredicción del modelo:')
    pred = predecir(objetivo, orient, [0.0] * 6, args.semillas)
    if not pred:
        print('  No encontré ninguna configuración dentro de JOINT_LIMITS para
ese objetivo y esa')
        print('  orientación. Prueba otro punto más cerca de la base o más
alto.')
    for i, d in enumerate(pred, 1):
        print(f'  rama {i}: q3 {d["signo_q3"]}  q(deg)={d["q_deg"]}  '
              f'desplazamiento desde la base: suma={d["desp_suma_rad"]} rad  '
              f'holgura a límites={d["holgura_limites_rad"]} rad')
    if args.solo_planear:
        print('\n--solo-planear: no se movió el brazo.')
        return

#1) conectar y llevar a la base
    print('\n' + LINEA)
    print('       sync_plan_nx NO debe estar corriendo (el puerto serie es
exclusivo).')
    if not args.si:
        input('ENTER para continuar, Ctrl+C para cancelar... ')
    brazo = conectar(args.puerto, args.baud, args.simular)

    print('\n[1] Yendo a la base (todos los ángulos en 0)...')
    q_base_deg, ok_base = ir_a_base(brazo, args.velocidad)
    if q_base_deg is None:
        sys.exit('No pude leer los ángulos en la base.')
    print(f'    q base (grados): {q_base_deg}   ' + ('OK' if ok_base else '(¡no
llegó exacto a 0!)'))
    q_base = [math.radians(v) for v in q_base_deg]

#2) IK del firmware
    pedido = objetivo + orient
    print(f'\n[2] send_coords({[round(v, 1) for v in pedido]},
velocidad={args.velocidad}, '
          f'modo={args.modo})  <- la IK la resuelve el firmware')
    q_pre = leer(brazo.get_angles, 6)
    brazo.send_coords(pedido, args.velocidad, args.modo)
    q_deg = esperar_reposo(brazo)
    if q_deg is None:
        sys.exit('No pude leer los ángulos finales.')
    se_movio = q_pre is None or max(abs(a - b) for a, b in zip(q_deg, q_pre)) >=
0.5
    if not se_movio:
        print('\nOJO: el brazo NO se movió. El firmware no encontró solución para
ese objetivo con')
        print('esa orientación (fuera de alcance u orientación imposible). Prueba
otro punto o')
        print('cambia --orientacion. No se registra esta corrida.')
        if not args.no_volver:
            ir_a_base(brazo, args.velocidad)
        sys.exit(2)
    coords = leer(brazo.get_coords, 6)

#3) auditoría con NUESTRA FK
    q = [math.radians(v) for v in q_deg]
    fx, fy, fz = fk.fk(q)
    e_ik = dist((fx, fy, fz), objetivo)
    e_fw = e_fk = None
    print('\n' + LINEA)
    print('RESULTADOS')
    print(LINEA)
    print(f'q ejecutado (grados) : {[round(v, 2) for v in q_deg]}')
```

```
    print(f'pedido               : ({objetivo[0]:8.1f}, {objetivo[1]:8.1f},
{objetivo[2]:8.1f}) mm')
    print(f'fk(q)  (nuestra FK)  : ({fx:8.1f}, {fy:8.1f}, {fz:8.1f}) mm')
    print(f'error por eje        : dx={objetivo[0] - fx:+.1f}  dy={objetivo[1] -
fy:+.1f}  '
          f'dz={objetivo[2] - fz:+.1f} mm')
    print(f'ERROR CARTESIANO (pedido vs fk(q)) : {e_ik:.2f} mm   '
          + ('[≤ 10 mm: CUMPLE]' if e_ik <= 10.0 else '[> 10 mm: REVISAR]'))
    if coords:
        e_fw = dist(coords[:3], objetivo)
        e_fk = dist(coords[:3], (fx, fy, fz))
        print(f'  get_coords() firmware            : ({coords[0]:.1f},
{coords[1]:.1f}, {coords[2]:.1f}) mm  '
              f'orientación={[round(v, 1) for v in coords[3:6]]}')
        print(f'  error pedido vs get_coords()     : {e_fw:.2f} mm   (lo que el
firmware cree que hizo)')
        print(f'  error fk(q) vs get_coords()      : {e_fk:.2f} mm   (error de
NUESTRO modelo, ítem 1)')
    print(LINEA)

#4) la pregunta del codo
    ramas = None
    if not args.sin_ramas:
        print('\nAnálisis de la rama (codo): buscando otras configuraciones que
alcancen la misma pose...')
        ramas = ik.analizar_ramas(q, q_base, semillas=args.semillas)
        ik.imprimir_ramas(ramas)
        print(f'\n  Rama elegida por el firmware: q3
{ramas["ejecutada"]["signo_q3"]}. Su criterio interno no')
        print('  está documentado; lo anterior son hechos medidos, no la regla
del fabricante.')

#5) evidencia
    ahora = datetime.datetime.now().isoformat(timespec='seconds')
    datos = {
        'fecha': ahora, 'simulado': bool(args.simular), 'objetivo_mm': objetivo,
        'orientacion_deg': orient, 'q_inicial_deg': q_base_deg,
'q_ejecutado_deg': q_deg,
        'fk_q_mm': [round(fx, 2), round(fy, 2), round(fz, 2)],
        'get_coords': coords, 'error_pedido_vs_fk_mm': round(e_ik, 3),
        'error_pedido_vs_get_coords_mm': None if e_fw is None else round(e_fw,
3),
        'error_fk_vs_get_coords_mm': None if e_fk is None else round(e_fk, 3),
        'prediccion_ramas': pred, 'ramas': ramas,
    }
    with open(args.json, 'w') as f:
        json.dump(datos, f, indent=2, ensure_ascii=False)
    nuevo = not os.path.exists(args.salida)
    with open(args.salida, 'a', newline='') as f:
        w = csv.writer(f)
        if nuevo:
            w.writerow(['fecha', 'simulado', 'x', 'y', 'z', 'rx', 'ry', 'rz',
'q1_deg', 'q2_deg',
                        'q3_deg', 'q4_deg', 'q5_deg', 'q6_deg', 'fk_x', 'fk_y',
'fk_z',
                        'error_mm', 'error_vs_get_coords_mm',
'error_fk_vs_get_coords_mm',
                        'rama_q3', 'n_configuraciones'])
        w.writerow([ahora, int(bool(args.simular))] + pedido + [round(v, 3) for v
in q_deg] +
                   [round(fx, 2), round(fy, 2), round(fz, 2), round(e_ik, 3),
                    '' if e_fw is None else round(e_fw, 3), '' if e_fk is None
else round(e_fk, 3),
                    '+' if q[2] >= 0 else '-', '' if ramas is None else
ramas['n_soluciones_distintas']])
    print(f'\nRegistrado en {args.salida} y {args.json}')
```

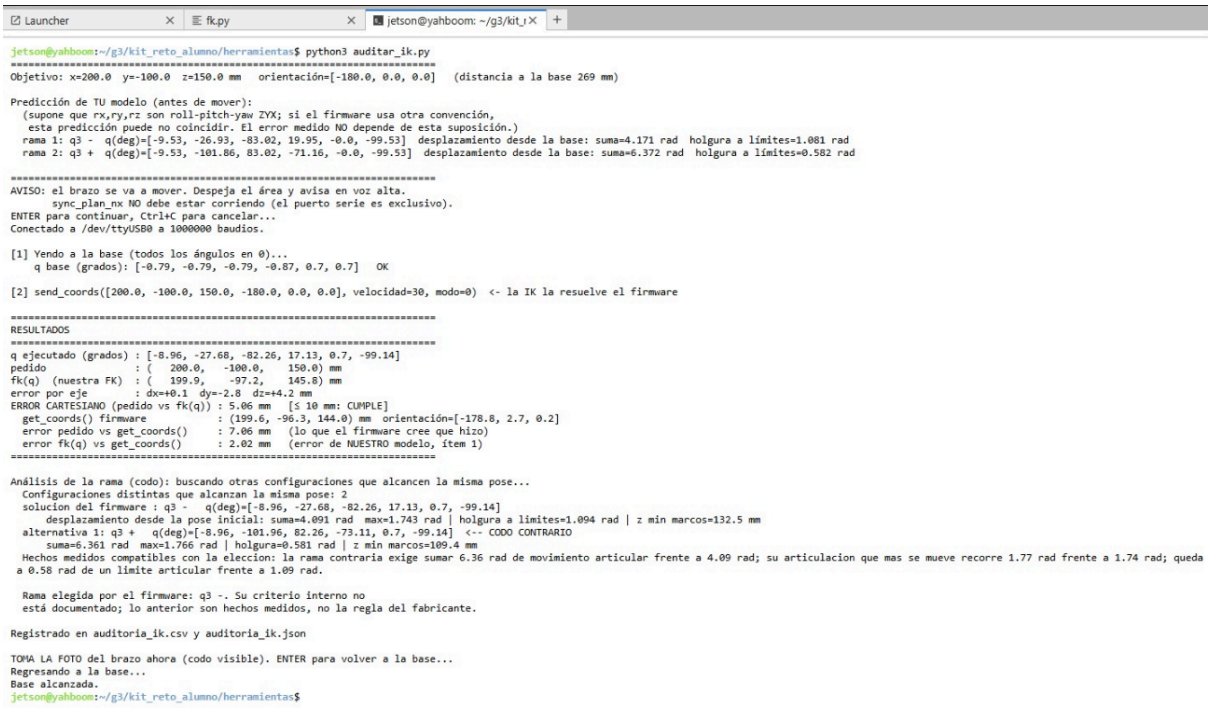
```
    #6) foto y regreso a la base
    if not args.si:
        input('\nTOMA LA FOTO del brazo ahora (codo visible). ENTER para volver a
la base... ')
    if not args.no_volver:
        print('Regresando a la base...')
        _, ok = ir_a_base(brazo, args.velocidad)
        print('Base alcanzada.' if ok else 'OJO: no quedó exacto en la base.')

if __name__ == '__main__':
    main()
```

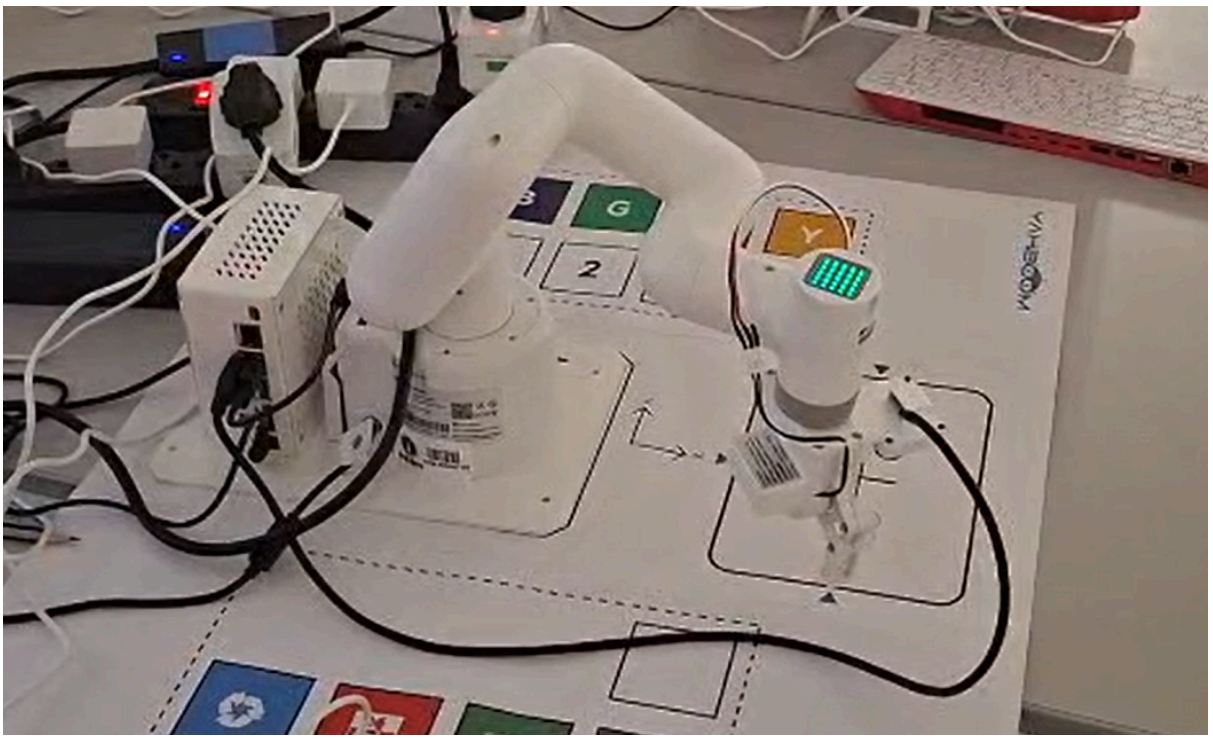
3. Análisis de la Rama Cinemática

En cinemática inversa, un mismo punto espacial puede alcanzarse mediante múltiples configuraciones articulares, como las posturas de "codo arriba" o "codo abajo".

Evidencia:



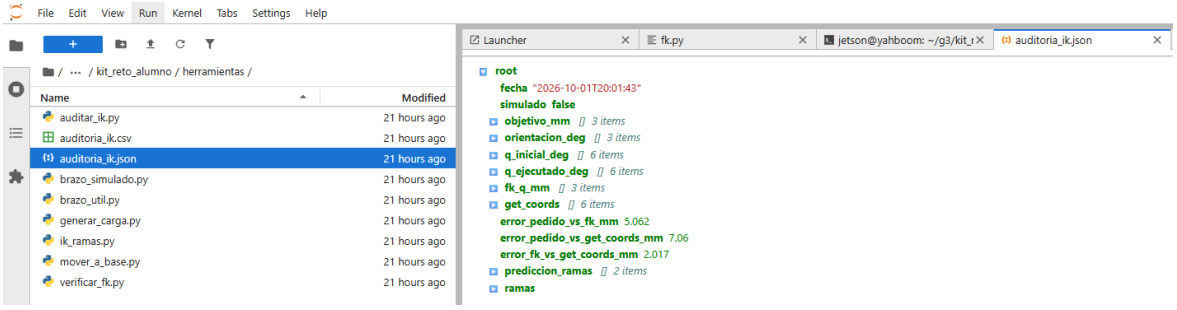
Resultados al ejecutar el codigo de auditar_ik.py



Posición del Jetcobot al ejecutar el comando python3 auditar_ik.py

fecha	x	y	z	rx	ry	rz	q1_deg	q2_deg	q3_deg	q4_deg	q5_deg	q6_deg	fk_x	fk_y	fk_z	error_mm	error_vs_get_coords_mm	error_fk_vs_get_coords_mm	rama_q3	n_configuraciones
2026-10-01 20:00:13	200.0	-100.0	150.0	-180.0	0.0	0.0	-8.96	-27.68	-82.26	17.22	0.7	-99.49	199.97	-97.23	145.89	4.96	6.886	1.944	-	2
2026-10-01 20:01:43	200.0	-100.0	150.0	-180.0	0.0	0.0	-8.96	-27.68	-82.26	17.13	0.7	-99.14	199.88	-97.21	145.78	5.062	07.06	2.017	-	2

Tabla csv generada para llevar el registro



Auditoria_ik.json